



Model Context Protocol

How AI applications discover and call external tools — one standard protocol instead of one integration per app per service

WHAT'S INSIDE

1. Why MCP — the N×M integration problem
2. What MCP actually is — hosts, clients & servers
3. The message format — JSON-RPC 2.0, tools/list & tools/call
4. Beyond tools — Resources & Prompts
5. One real request, message by message
6. MCP in Claude Code — .mcp.json, scopes & transports
7. MCP vs. direct API integration, glossary & self-test

Why MCP — the N×M Integration Problem

Every AI application wiring itself to every service, one custom integration at a time

Say you're building an AI application that needs to work with Azure DevOps, GitHub, and Jira. Each one has its own API: its own authentication scheme, its own endpoints, its own request and response formats. To call any of them, the application needs custom integration code written specifically for that service.

```
# Custom code, written for Azure DevOps specifically
def get_azure_devops_bug(bug_id: int) -> dict:
    url = f"https://dev.azure.com/{ORG}/{PROJECT}/_apis/wit/workitems/{bug_id}?api-version=7.1"
    r = requests.get(url, auth="", os.environ["AZURE_DEVOPS_PAT"])
    fields = r.json()["fields"]
    return {"title": fields["System.Title"]}

# GitHub needs its own URL, its own auth, its own field names
# Jira needs a third, completely different version of the same thing
```

Two costs that multiply, not add

DEVELOPMENT EFFORT

The developer has to learn each service's authentication, endpoints, request formats, response formats, and schemas — separately, for every single service.

MAINTENANCE

These integrations aren't write-once. When Azure DevOps, GitHub, or Jira changes its API, the integration code has to be updated to keep working — for every app that built its own copy.

Now add a second AI application — a chat assistant, say — that also needs Azure DevOps, GitHub, and Jira. It writes its own three integrations from scratch. A third application does the same. Three apps, three services: **9 separate integrations**, each maintained independently.

THE ACTUAL SHAPE OF THE PROBLEM

This isn't a fixed cost — it's N×M. Every new AI application multiplies by every existing service; every new service multiplies by every existing application. Add a fourth service and every app needs a new integration. Add a fourth app and it needs to rebuild every integration that already exists.

What MCP Actually Is

One integration per service, written once, reused by every application

The Model Context Protocol (MCP) is an open standard that defines how AI applications discover and call external tools and data sources. Instead of every AI application writing its own integration code for every service, the integration lives in one place: an **MCP server**.

🔗 Three roles in the architecture



Host
The AI application itself — Claude Code, a chat assistant, an IDE copilot. It's what the person actually uses.



Client
Lives inside the host. Speaks MCP to exactly one server, maintaining that connection and translating between the host and the server.



Server
A lightweight program that exposes a specific service's capabilities — tools, data, templates — through the MCP protocol.

🔄 Before and after

WITHOUT MCP	WITH MCP
Each AI application writes integration code <i>inside itself</i> for every service it needs — the HTTP calls, the auth, the response parsing, all hand-written per app, per service.	Each service gets one MCP server, written once, that holds the integration code. Any MCP-capable application connects to it with the same protocol.
3 apps × 3 services = 9 integrations , each maintained separately.	3 services = 3 servers , written once, plus one MCP client implementation per app. The work adds up instead of multiplying.

A FAMILIAR ANALOGY

If you've worked with Selenium, think of WebDriver. Your test script doesn't know Chrome's internals — it speaks one protocol to a driver, and the driver handles the browser-specific details. MCP plays that same role between an AI application and a tool: the application speaks one protocol, and the server handles everything specific to the underlying service.

WHAT DOESN'T CHANGE

MCP doesn't replace the Azure DevOps, GitHub, or Jira REST APIs — the server still calls those APIs underneath. What MCP replaces is the need for *every application* to know how to call them directly.

The Message Format & Tool Operations

A common structure for every message, and two standard operations for working with tools

For the client and server to understand each other, MCP needs a shared message format. MCP uses **JSON-RPC 2.0** for every protocol message — a lightweight standard that gives both sides a common structure for requests and responses, so no client or server has to invent its own.

✉ A request and its response

```
// Client → Server: run the get_bug tool
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_bug",
    "arguments": { "bug_id": 123 }
  }
}

// Server → Client: same id, so the client knows which request this answers
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [{ "type": "text", "text": "Bug 123: Login button not working" }],
    "isError": false
  }
}
```

FIELD	MEANING
jsonrpc	Which JSON-RPC version is in use — always "2.0"
id	Identifies this specific request; the response echoes it back so the client can match answers to questions
method	The operation being requested — e.g. <code>tools/call</code>
params	The operation's details — here, the tool name and its arguments
isError	Lets the model tell a real failure from a real answer, without the whole request erroring out

🔍 Two standard operations

TOOLS/LIST

"Hey server, what tools do you have?" The server replies with each tool's name, a description, and an input schema (JSON Schema) describing the parameters it expects.

TOOLS/CALL

"Run this specific tool with these arguments." The server executes it, does whatever work the real integration requires, and returns the result.

SWAP THE SERVER, KEEP THE CLIENT CODE

Replace the Azure DevOps MCP server with a GitHub MCP server and the client still sends exactly `tools/list` and `tools/call`. Only the tool names and their schemas change — not the mechanism the application uses to talk to the server.

Beyond Tools: Resources & Prompts

MCP servers expose three building blocks — tools are only one of them

Tools get most of the attention because they're the piece that lets a model take action. But the official MCP specification defines three distinct server building blocks, each controlled by a different party in the conversation. Knowing all three explains what an MCP server can offer beyond "call this function."

FEATURE	CONTROLLED BY	WHAT IT'S FOR	EXAMPLE
Tools	The model	Functions the model actively decides to call, based on the user's request. Can write to a database, call an external API, or modify files.	<code>searchFlights()</code>
Resources	The application	Passive, read-only data the application retrieves and decides how to use — file contents, a database schema, documentation. The model doesn't call these; the host fetches and hands them over.	<code>calendar://events/2026</code>
Prompts	The user	Pre-built, parameterized instruction templates the user explicitly invokes — a structured starting point for a task, often exposed as a slash command.	<code>/plan-vacation</code>

🔗 Resources: direct vs. templated

DIRECT RESOURCE

A fixed URI pointing at specific data. `calendar://events/2026` always returns the same thing: this year's calendar.

RESOURCE TEMPLATE

A parameterized URI for flexible queries, e.g. `weather://forecast/{city}/{date}` . Supports parameter completion — typing "Par" can suggest "Paris."

🔗 Protocol operations for each

FEATURE	DISCOVER	USE
Tools	<code>tools/list</code>	<code>tools/call</code>
Resources	<code>resources/list</code> & <code>resources/templates/list</code>	<code>resources/read</code>
Prompts	<code>prompts/list</code>	<code>prompts/get</code>

HOW THEY COMBINE

A single request can use all three: a user invokes a `plan-vacation` prompt, the application attaches calendar and travel-history *resources* as context, and the model calls `searchFlights` and `checkWeather` *tools* to carry it out — three mechanisms, one coordinated task.

One Real Request, Message by Message

"Find bug 123 in Azure DevOps" — twelve messages behind one question and one answer

From the outside this looks like one question and one answer. Underneath, twelve messages cross four boundaries: the user to the host, the host to the model, the host to the MCP server, and the server to the real Azure DevOps REST API.

#	PHASE	MESSAGE	WHAT'S HAPPENING
1	Discovery (at startup)	<code>tools/list</code>	The MCP client asks the server what tools it offers, before any question is typed
2		<code>tool definitions</code>	Server replies with <code>get_bug</code> 's name, description, and input schema
3	Model picks a tool	"Find bug 123..."	The user's plain-text question reaches the host application
4		<code>prompt + tool list</code>	The host sends the model the question <i>and</i> the tool definitions it collected
5		<code>tool_use get_bug(123)</code>	The model doesn't call anything itself — it returns a structured request for the tool
6	The tool call	<code>tools/call get_bug(123)</code>	The client forwards this as a standard MCP <code>tools/call</code> message
7		<code>GET workitems/123</code>	Only now does Azure DevOps get involved — an ordinary REST call using the server's own credentials
8		<code>work item JSON</code>	Azure DevOps replies in its own format, with its own field names
9	The answer	<code>result: content[text]</code>	The server wraps the REST response as a standard MCP tool result
10		<code>tool_result</code>	The host hands the result to the model in the model API's own format — MCP ends at the host
11		"Bug 123 is titled..."	The model writes the final reply in plain language
12		Shown in the chat	The host displays it — one question, one answer, from the user's side

THE ONE LINE WORTH REMEMBERING

The model never sees JSON-RPC, and MCP never sees the user's final answer. MCP covers exactly the middle section — between the host's MCP client and the server — while the model API and the real service's REST API each keep their own separate message formats either side of it.

HUMAN OVERSIGHT IS PART OF THE SPEC

Before step 6 actually fires, most hosts pause for your approval — MCP's own guidance is that there should always be a human able to deny a tool call before it runs, especially for anything with side effects.

MCP in Claude Code

Connecting a server with `.mcp.json`, and what shows up once it's connected

`.mcp.json` at the project root

```
{
  "mcpServers": {
    "azure-devops": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@azure-devops/mcp-server"],
      "env": { "ADO_PAT": "${ADO_PAT}" }
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@playwright/mcp-server"]
    }
  }
}
```

Committing `.mcp.json` shares these servers with the whole team — anyone who opens the project gets the same MCP servers available, the same way `CLAUDE.md` shares project instructions.

Transports: how the client reaches the server

TRANSPORT	HOW IT CONNECTS
<code>stdio</code>	Launches the server as a local process and talks over standard input/output — the common choice for a server running on your own machine
<code>http</code>	Connects to a remote server over HTTP — the recommended choice for hosted/remote servers
<code>sse</code>	Server-Sent Events — still supported, now considered the deprecated predecessor to HTTP

Scopes: where a server is available

SCOPE	STORED IN	LOADS IN
Local	<code>~/.claude.json</code>	This project only, this machine only — not shared
Project	<code>.mcp.json</code>	This project, shared with the team via git
User	<code>~/.claude.json</code>	Every project on this machine

How a connected server's tools show up

Once connected, every tool from that server is prefixed with the server's name, so Claude (and you) can tell at a glance which server a tool came from:

```
mcp_azure-devops_get_bug           # from the azure-devops server
mcp_azure-devops_create_test_case  # from the azure-devops server
mcp_playwright_navigate            # from the playwright server
mcp_playwright_click               # from the playwright server
```

THE TWO EXAMPLES FROM THIS SERIES

An Azure DevOps MCP server lets the Test Case Generator pull a real PBI or bug straight from ADO instead of a pasted user story. A Playwright MCP server lets Claude drive a real browser — navigate, click, read the page — directly inside the automation framework, using the same `tools/list` / `tools/call` mechanism either way.

MCP vs. Direct API Integration, & Self-Test

What changes, what doesn't, and how to check your understanding

	DIRECT API INTEGRATION	MCP
Who writes the integration	Every application, for every service, separately	One MCP server per service, written once
What the model sees	Whatever shape the developer hand-wrote into the tool definition	Tool name, description, and JSON Schema — discovered via <code>tools/list</code>
Adding a new service	New integration code in every application that needs it	One new server; every MCP-capable application can use it immediately
Does it replace the service's REST API?	N/A	No — the server still calls that REST API underneath

THEY COMPOSE, NOT COMPETE

MCP doesn't remove the need for a real API integration somewhere — it just concentrates that integration into one reusable server, instead of duplicating it inside every AI application that needs it.

Glossary

TERM	MEANING
Host	The AI application itself — e.g. Claude Code, a chat assistant
Client	The part inside the host that speaks MCP to one specific server
Server	The program exposing a service's tools, resources, and prompts via MCP
JSON-RPC 2.0	The common message structure every MCP request and response follows
Tool	A model-controlled function the AI can call to take an action
Resource	Application-controlled, read-only data the host retrieves for context
Prompt	A user-controlled, parameterized instruction template

Self-test

Q1

Why is the integration problem $N \times M$ rather than just a fixed cost that grows slowly?

Q2

What does `tools/list` return, and when does a client typically call it?

Q3

Who controls a Tool, who controls a Resource, and who controls a Prompt?

Q4

In the 12-message flow, at which step does Azure DevOps' own REST API actually get called?

Q5

What changes and what stays the same when you swap one MCP server for another?